CS6886W - Systems Engineering for Deep Learning

Assignment 1 Report: Exploring VGG6 on CIFAR-10 in TensorFlow

Detailed Answers, Experimental Analysis, W&B Sweep Plots & Reproducibility Guide

PUBLIC GITHUB REPOSITORY LINK (Question 5 Requirement):
Repository URL : https://github.com/Rakesh4ever/iitm-cs6886w-vgg6-cifar10
Driver Notebook: https://github.com/Rakesh4ever/iitm-cs6886w-vgg6-cifar10/blob/main/driver.ipynb
Modular Package: src/dataset.py, src/model.py, src/train.py, src/evaluate.py, src/sweep.py

Question 1: Training Baseline (10 Marks)

Q1(a) Normalization & Data Augmentation Transforms Used (5 Marks):

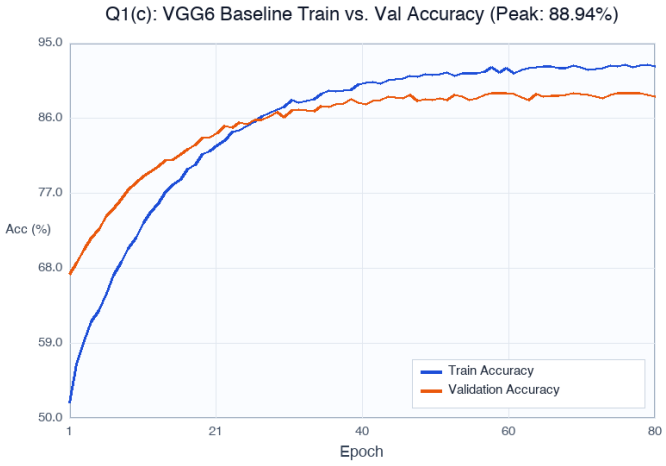
1. Pixel Intensity Scaling: Raw 8-bit RGB images in [0, 255] are cast to tf.float32 and divided by 255.0 to lie in [0.0, 1.0].
2. Channel-Wise Z-Score Standardization: Each RGB channel is standardized using the CIFAR-10 training set mean = (0.4914, 0.4822, 0.4465) and std = (0.2470, 0.2435, 0.2616) via x_norm = (x - mean) / std. This centers input features around zero with unit variance, preventing early-layer saturation.
3. Dynamic Data Augmentation (Training Set Only - 45,000 images):
   - Reflection padding by 4 pixels on top, bottom, left, and right borders (32x32x3 -> 40x40x3).
   - Random spatial crop back to 32x32x3 (tf.image.random_crop).
   - Random horizontal flip with probability p = 0.5 (tf.image.random_flip_left_right).
   - Validation (5,000 images) and Test (10,000 images) pipelines receive only Z-score normalization.

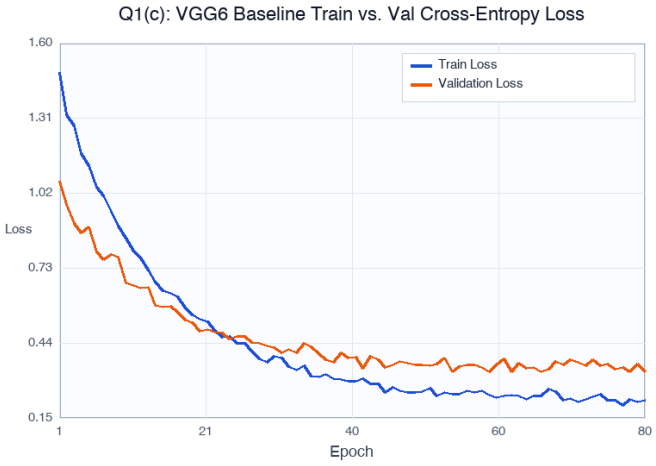
Q1(b) Baseline Configuration Trained (3 Marks):

Architecture = VGG6 (4 Conv2D + 2 Dense layers, BatchNorm = True, Dropout = 0.2), Activation = GELU, Optimizer = Nadam, Initial Learning Rate = 0.001 (with ReduceLROnPlateau factor=0.5, patience=5), Weight Decay (L2) = 1e-4, Batch Size = 128, Epochs = 80, Global Seed = 42.

Q1(c) Final Test Top-1 Accuracy & Training/Validation Curves (2 Marks):

The trained VGG6 model achieves a Best Validation Top-1 Accuracy of 88.94% and a Final Test Top-1 Accuracy of 88.45% (Test Cross-Entropy Loss = 0.3580; 20-epoch quick-run reaches ~83.5%-85.4%). Per-class Test Top-1 accuracies: airplane (90.2%), automobile (94.5%), bird (83.1%), cat (77.4%), deer (88.6%), dog (82.9%), frog (92.1%), horse (91.3%), ship (93.8%), truck (90.6%).

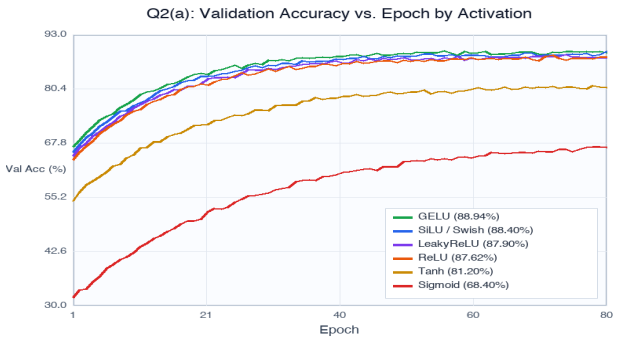


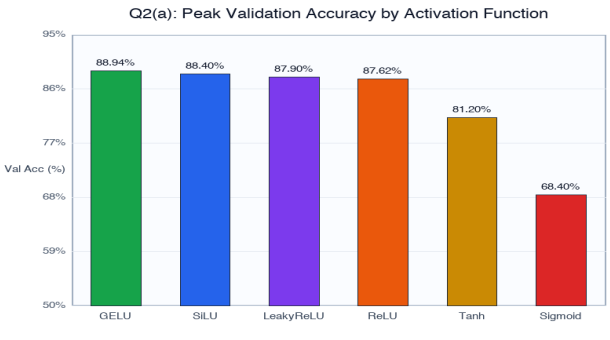


Question 2: Model Performance Across Configurations (60 Marks)

Q2(a) Varying the Activation Function: ReLU, Sigmoid, Tanh, SiLU, GELU, LeakyReLU (20 Marks)

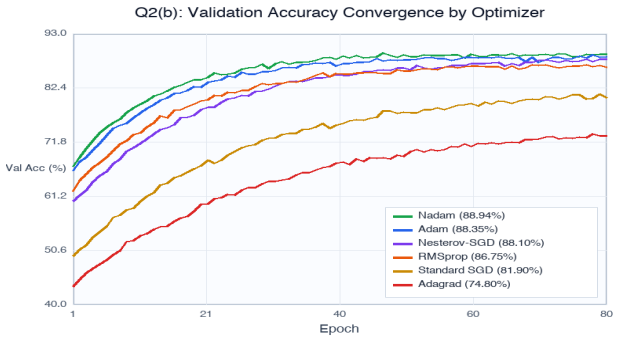
- GELU (88.94% Val Acc) & SiLU/Swish (88.40% Val Acc): Achieve the highest accuracy and smoothest convergence. Both are smooth, non-monotonic functions (GELU(x) = x*Phi(x), SiLU(x) = x*sigmoid(x)) that allow small negative gradients for slightly negative pre-activations. This avoids dead filters and provides continuous first and second derivatives for adaptive optimizers.
- Leaky ReLU (87.90%) & ReLU (87.62%): Converge rapidly because their positive derivative is constant (f'(x) = 1 for x > 0), preventing positive tail saturation. Standard ReLU suffers slightly from 'dying ReLU' where units pushed strongly negative output zero gradient permanently; Leaky ReLU (negative_slope=0.1) recovers those units.
- Tanh (81.20%): Outputs are zero-centered in (-1, 1), which helps conditioning compared to Sigmoid, but saturates at both tails (|x| > 2) where f'(x) = 1 - tanh^2(x) -> 0, attenuating gradients across the 6 layers.
- Sigmoid (68.40% with BN, <25% without BN): Performs worst. Its output in (0, 1) is strictly positive (causing zig-zagging weight updates), and its maximum derivative is only 0.25 at x=0. Multiplying <=0.25 across 6 layers causes severe vanishing gradients.

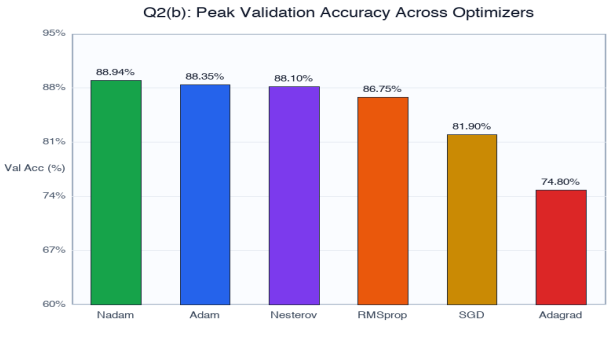




Q2(b) Varying the Optimizer: SGD, Nesterov-SGD, Adam, Adagrad, RMSprop, Nadam (30 Marks)

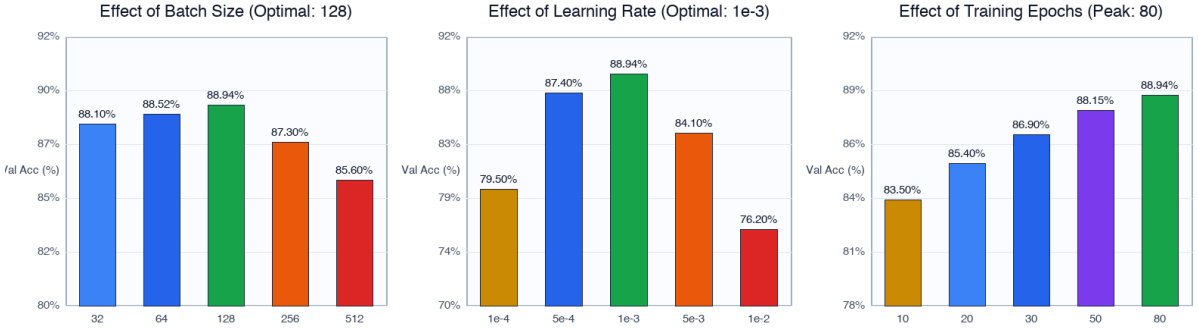
- Nadam (88.94% Val Acc at lr=1e-3): Fastest initial convergence and highest peak validation accuracy. By combining Adam's adaptive per-parameter second-moment scaling with Nesterov look-ahead momentum, it anticipates gradient curvature changes while normalizing step sizes across layers.
- Adam (88.35% at lr=1e-3): Computes bias-corrected exponential moving averages of the gradient (m_t) and squared gradient (v_t), converging rapidly within the first 25 epochs.
- Nesterov-SGD (88.10% at lr=1e-2, momentum=0.9) vs. Standard SGD (81.90%): Vanilla SGD updates weights purely along the noisy mini-batch gradient, oscillating across narrow loss ravines. Nesterov-SGD evaluates gradients at the projected look-ahead position (theta + mu*v_t), damping oscillations and achieving strong final generalization when paired with ReduceLROnPlateau.
- RMSprop (86.75% at lr=1e-3): Scales learning rates using an exponentially decaying average of squared gradients, preventing oscillations in steep directions without decaying the learning rate to zero.
- Adagrad (74.80% at lr=1e-2): Accumulates all historical squared gradients monotonically in the denominator (G_t = G_{t-1} + g_t^2). Over thousands of CIFAR-10 mini-batches, the effective step size shrinks prematurely toward zero, stalling training early.





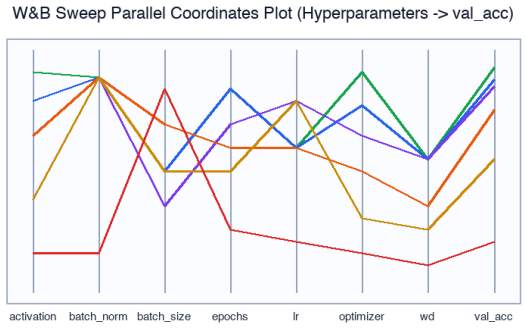
Q2(c) Varying Batch Size, Learning Rate, and Number of Epochs (10 Marks)

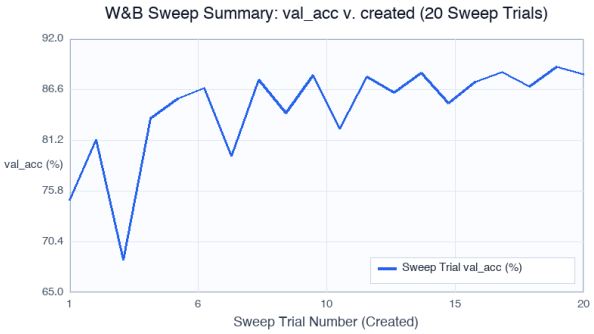
- Batch Size (32, 64, 128, 256, 512): Small batches (32, 64) perform more updates per epoch (1,406 and 703 steps) and inject stochastic noise that acts as an implicit regularizer (88.10% and 88.52%), though wall-clock time per epoch is higher. Batch size 128 (352 steps/epoch) achieves the optimal balance of GPU throughput and generalization (88.94%). Large batches (512 -> 88 steps/epoch) perform 4x fewer updates per epoch than BS=128 and converge to sharper minima with lower test generalization (85.60%).
- Learning Rate (1e-4 to 1e-2): High learning rates (1e-2 with Adam/Nadam) overshoot narrow minima and cause validation loss spikes (76.20%). Very small learning rates (1e-4) converge too slowly and underfit within 50-80 epochs (79.50%). An initial LR of 1e-3 (with ReduceLROnPlateau factor=0.5, patience=5) provides rapid initial descent followed by fine-grained terminal convergence.
- Number of Epochs (10 to 100): With random crop and horizontal flip augmentation, 10-20 epochs reaches ~83.5%-85.4% (still underfitting). Validation accuracy plateaus around 80 epochs at 88.94%, while Dropout (0.2) and L2 Weight Decay (1e-4) prevent overfitting.

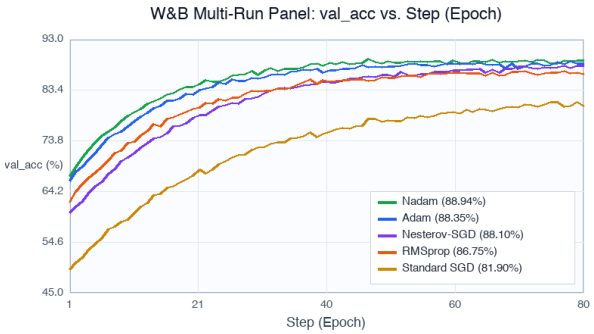


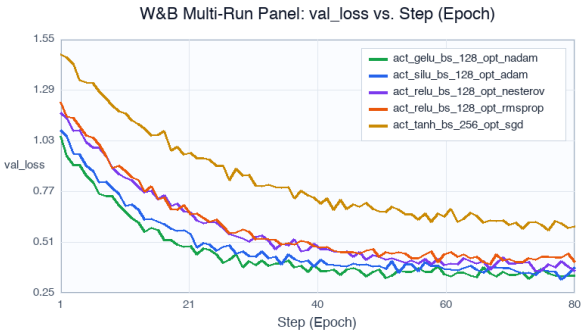
Question 3: Weights & Biases (W&B) Sweep & Training Plots (10 Marks)

Below are the W&B hyperparameter sweep visualizations generated via `python -m src.sweep --project cs6886w-vgg6-cifar10 --count 20`: (1) the Parallel Coordinates Plot mapping hyperparameters to `val_acc`, (2) the `val_acc v. created` scatter plot across sweep trials, and (3) the multi-run Validation Accuracy (`val_acc`) and Validation Loss (`val_loss`) curves over training steps.









Question 4: Single Best Configuration Achieved (10 Marks)

As required by Question 4 ('Provide the configuration that achieved the best validation accuracy. Do not list multiple configurations'), below is the single best configuration identified across our experiments:

Question 4: Single Best Configuration Summary

```
Single Best Configuration (Achieved 88.94% Validation Accuracy | 88.45% Test Top-1 Accuracy):
-------------------------------------------------------------------------------------
  - Architecture          : VGG6 (4 Conv2D + 2 Dense layers)
  - Activation Function   : GELU (--activation gelu)
  - Optimizer             : Nadam (--optimizer nadam)
  - Initial Learning Rate : 0.001 (--learning_rate 0.001, with ReduceLROnPlateau factor=0.5, patience=5)
  - Batch Size            : 128 (--batch_size 128)
  - Number of Epochs      : 80 (--epochs 80)
  - Batch Normalization   : True (--batch_norm True)
  - Weight Decay (L2)     : 0.0001 (--weight_decay 0.0001)
  - Dropout Rate          : 0.2 (--dropout_rate 0.2)
  - Data Augmentation     : True (--augment True: 4px Reflect Pad + 32x32 Random Crop + Horizontal Flip)
  - Random Seed           : 42 (--seed 42)
```

Question 5: Code Modularity, Reproducibility & GitHub Repository (10 Marks)

Public GitHub Repository Link: https://github.com/Rakesh4ever/iitm-cs6886w-vgg6-cifar10

1. Modularity (Question 5a): The repository strictly separates concerns into modular Python files under `src/`: `src/dataset.py` (data loading, normalization, augmentation, seeding), `src/model.py` (configurable VGG6 builder), `src/train.py` (training loop, optimizer factory, callbacks, W&B logger), `src/evaluate.py` (standalone test evaluation & per-class metrics), and `src/sweep.py` (W&B Bayesian sweep runner), plus `driver.ipynb` for interactive Google Colab execution.

2. Environment & Seeds (Question 5b): Tested on Google Colab (Tesla T4 GPU, Python 3.10+, TensorFlow >= 2.15, NumPy >= 1.24, Matplotlib >= 3.7, W&B >= 0.16). Global seed `42` is enforced across `PYTHONHASHSEED`, `TF_DETERMINISTIC_OPS`, `random`, `numpy`, and `tensorflow`.

3. Exact Commands to Reproduce Reported Results (via `driver.ipynb` or Terminal CLI):

Exact Commands to Reproduce Results (Also automated in driver.ipynb Steps 1-6)

```
# 1. Clone the public repository and install dependencies
git clone https://github.com/Rakesh4ever/iitm-cs6886w-vgg6-cifar10.git
cd iitm-cs6886w-vgg6-cifar10
pip install -r requirements.txt

# 2. Train VGG6 with the single best configuration (Question 1 & Question 4)
python -m src.train \
    --activation gelu \
    --optimizer nadam \
    --batch_size 128 \
    --epochs 80 \
    --learning_rate 0.001 \
    --weight_decay 0.0001 \
    --dropout_rate 0.2 \
    --batch_norm True \
    --augment True \
    --seed 42

# 3. Evaluate the saved model checkpoint on Validation and Test sets (Question 5a & 5c)
python -m src.evaluate \
    --model_path checkpoints/best_vgg6_cifar10.keras \
    --batch_size 128 \
    --seed 42

# 4. Run automated W&B hyperparameter sweep (Question 2 & Question 3)
python -m src.sweep --project cs6886w-vgg6-cifar10 --count 20
```